

Conductivité thermique d'un matériau non-homogène : influence de la porosité sur la conductivité thermique

Chouillou Thao

N°SCEI :11263

Positionnement thématique :

Introduction

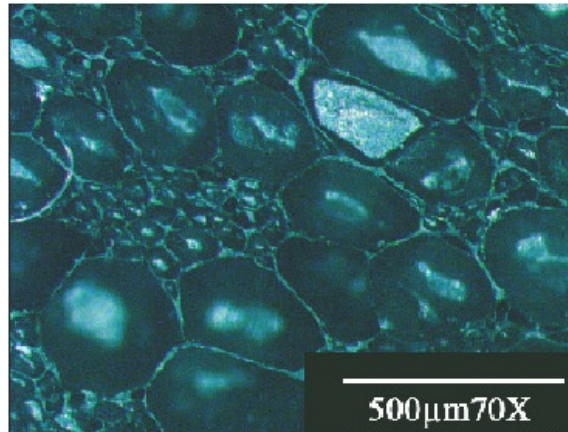
Lien avec le thème Sport et Jeux :

- Limiter les risques d'hypothermie (sécurité)
- Conserver l'énergie du corps (efficacité) = améliorer les performances
- Permettre les activités nautiques pendant toute l'année
- Eau conducteur de chaleur : 25 fois plus que l'air

Matériaux	Conductivité thermique ($\text{W.m}^{-1}.\text{K}^{-1}$)
Eau	0,598
Air	0,025
Néoprène	0,192
Caoutchouc	0,37

Introduction

Néoprène : Type de caoutchouc synthétique qui contient de nombreuses bulles d'air piégées à l'intérieur du matériau (sous forme de mousse)



Matériau non homogène : Matériau dont les propriétés physiques ou chimiques varient d'un point à un autre à l'intérieur du volume du matériau. (entraîne des variations des caractéristiques telles que la densité ou la conductivité thermique)

Problématique

Comment modéliser les flux thermiques dans un matériau non homogène pour comprendre le fonctionnement d'une combinaison en néoprène ?

Modélisation générale du problème

1. Modèle 1D pour simplifier le fonctionnement
2. Approfondissement du modèle en passant en 2D
3. Approche statistique

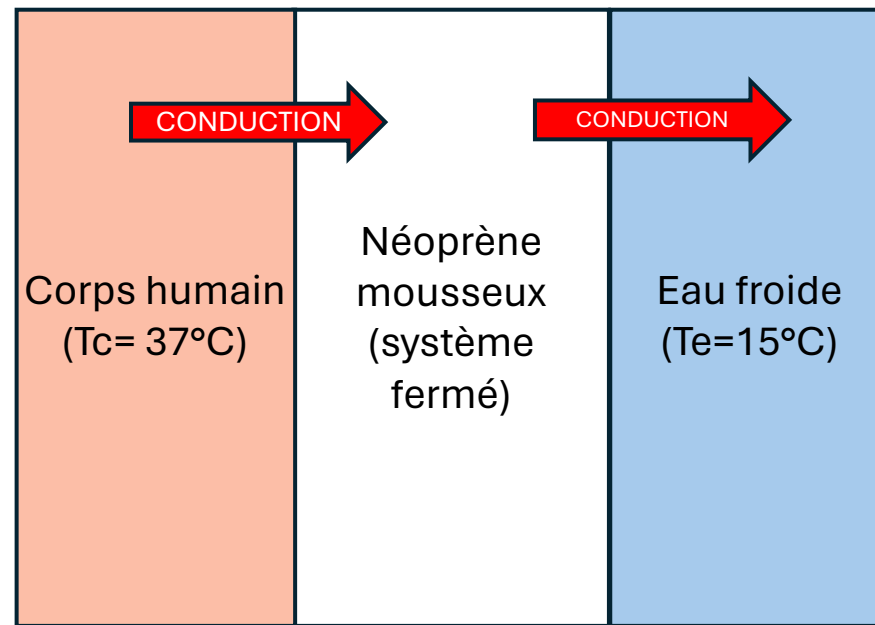
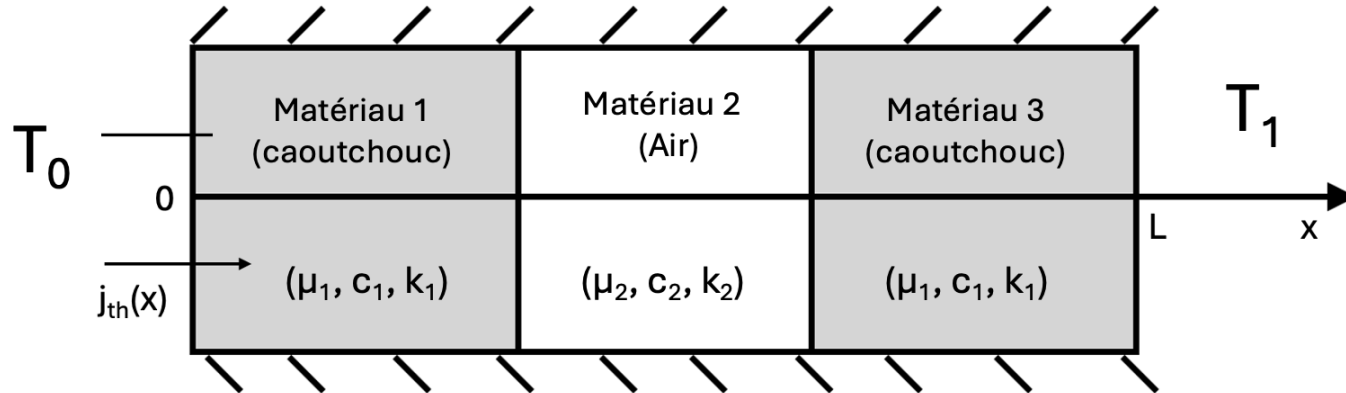


Figure : modélisation du problème et des transferts thermiques

1ère approche en 1D



Loi locale de la conservation d'énergie : $\frac{\partial u_v}{\partial t} + \text{div}(\vec{j}_{th}) = 0$

Loi de Fourier : $\vec{j}_{th} = -k \overrightarrow{\text{grad}}(T)$

Equation de la chaleur 1D :

$$\text{div}(k \overrightarrow{\text{grad}}(T)) = \mu c \frac{\partial T}{\partial t}$$

Adimensionnement

Introduction de variables adimensionnées :

Longueur :

$$X = \frac{x}{L}$$

Temps:

$$\tau = t / \frac{L^2}{D_{th}}$$

$$D_{th} = \frac{k}{\mu c}$$

Coefficient de
diffusion
thermique

Température :

$$\theta = \frac{T - T_0}{T_1 - T_0}$$

$$\Rightarrow \theta(X = \frac{x}{L}, \tau = t / \frac{L^2}{D_{th}})$$

Paramètres du
matériau :

$$\begin{cases} \mu = \mu_0 \tilde{\mu} \\ c = c_0 \tilde{c} \\ k = k_0 \tilde{k} \end{cases}$$

μ_0, c_0, k_0 des valeurs de références
propres au matériau

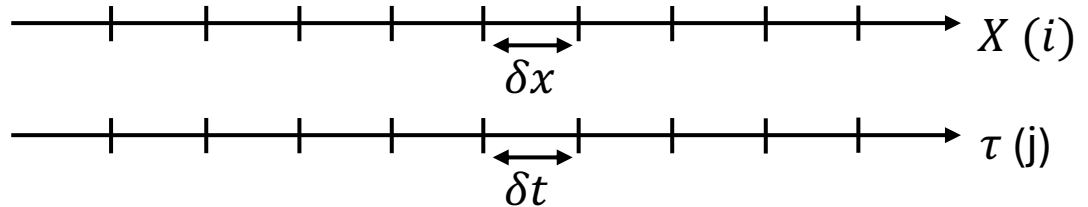
Equation de la chaleur adimensionnée :

$$\tilde{\mu} \tilde{c} \frac{\partial \theta}{\partial \tau} = \frac{\partial}{\partial X} (\tilde{k} \frac{\partial \theta}{\partial X})$$

Méthode d'Euler

Discrétisation de l'espace et du temps :

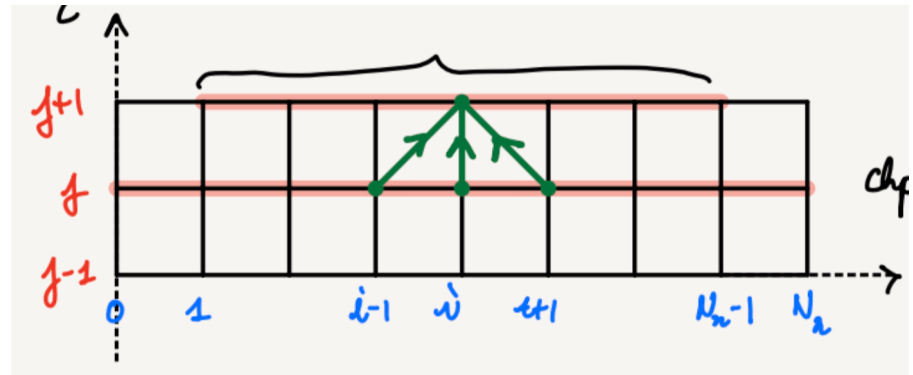
$$\theta_{ij} = \theta(X_i = i\delta x, \tau_j = j\delta t)$$



Approximation par développement de Taylor :

$$\frac{\partial \theta_{ij}}{\partial \tau} = \frac{\theta_{i,j+1} - \theta_{i,j}}{\delta t} + o(\delta t)$$

$$\frac{\partial \theta_{ij}}{\partial X} = \frac{\theta_{i+1,j} - \theta_{i,j}}{\delta x} + o(\delta x)$$



$$\theta_{i,j+1} = \frac{1}{\tilde{\mu}\tilde{c}} \frac{\partial t}{\partial x} \left(\widetilde{k_{i+1}} \left(\frac{\theta_{i+1,j} - \theta_{i,j}}{\delta x} \right) - \tilde{k}_i \left(\frac{\theta_{i,j} - \theta_{i-1,j}}{\delta x} \right) + \theta_{i,j} \right)$$

Relation de récurrence

Régime Stationnaire

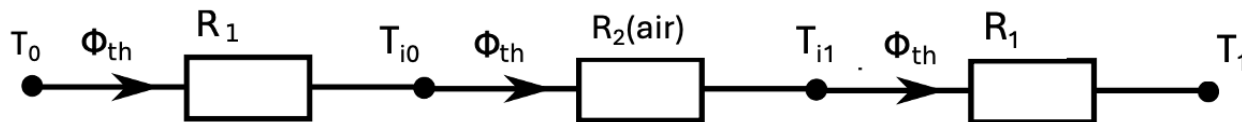
Temps caractéristique de diffusion thermique :

$$\tau \sim \frac{L^2}{D_{th}} = \frac{L^2 \mu c}{k} \quad 120 \text{ s} \ll 3600 \text{ s} = 1 \text{ h}$$

On se place en **RÉGIME STATIONNAIRE**

Régime Stationnaire:

$$\text{div}(\vec{j}_{th}) = 0 \Rightarrow \vec{j}_{th} \text{ à flux conservatif (uniforme en 1D)}$$



$$R_{th \text{ théorique}} = R_1 + R_2 + R_3$$

$$R_{i, \text{ surfacique}} = \frac{L}{k_i}$$

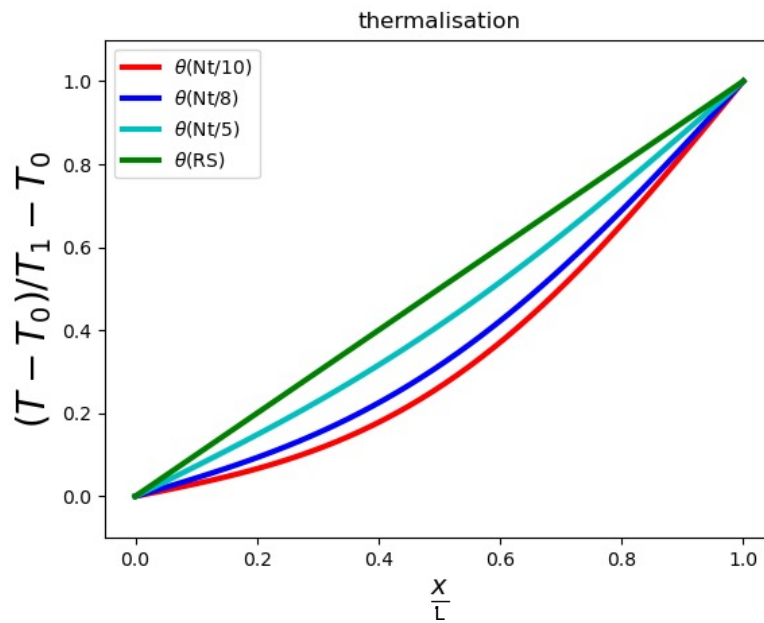
$$R_{th \text{ surfacique}} = \frac{L}{k_c j_{th}}$$

Allure de la température dans le matériau

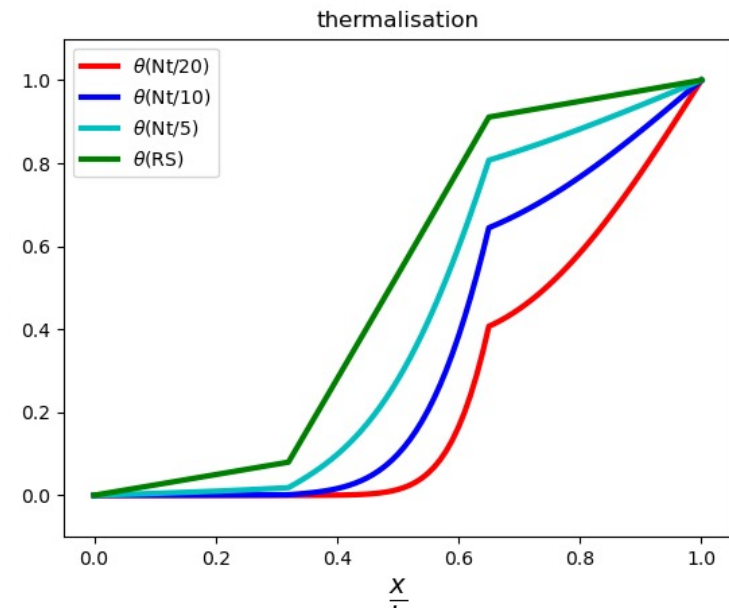
Résolution numérique :

Req = 0.040

Req_theorie = 0.045



$R_{th \text{ surfacique}} = 0.008 \text{ K.W}^{-1}.\text{m}^2$



$R_{th \text{ surfacique}} = 0.040 \text{ K.W}^{-1}.\text{m}^2$

La bande d'air permet d'améliorer l'isolation thermique en augmentant la résistance thermique totale

Limitations du modèle

Application en plongée :

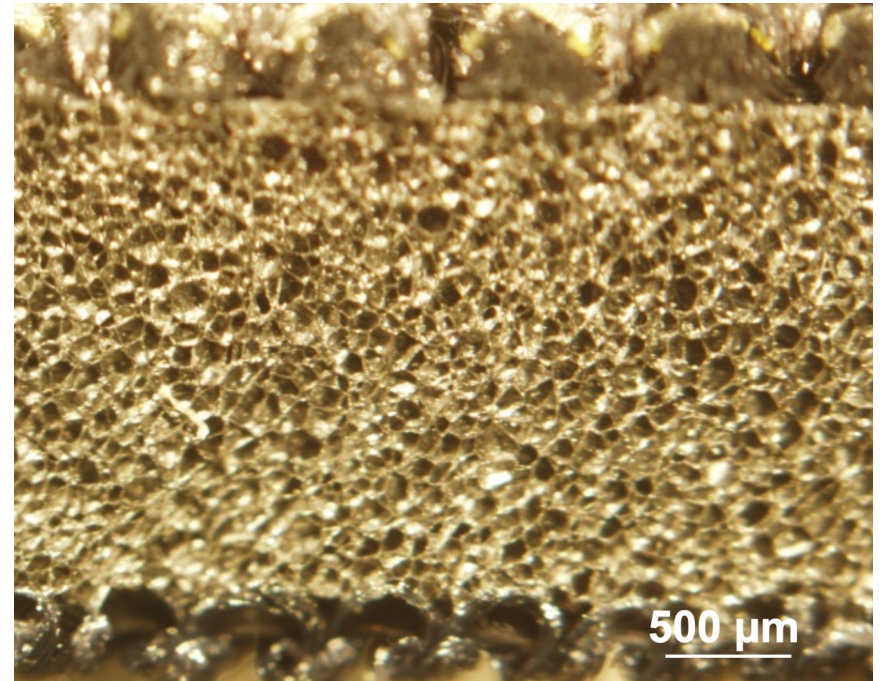
Combinaison étanche permettant de maintenir une couche d'air entre la peau et la combinaison

Contraintes :

- compliqué à enfiler
- fragile
- peu de maniabilité

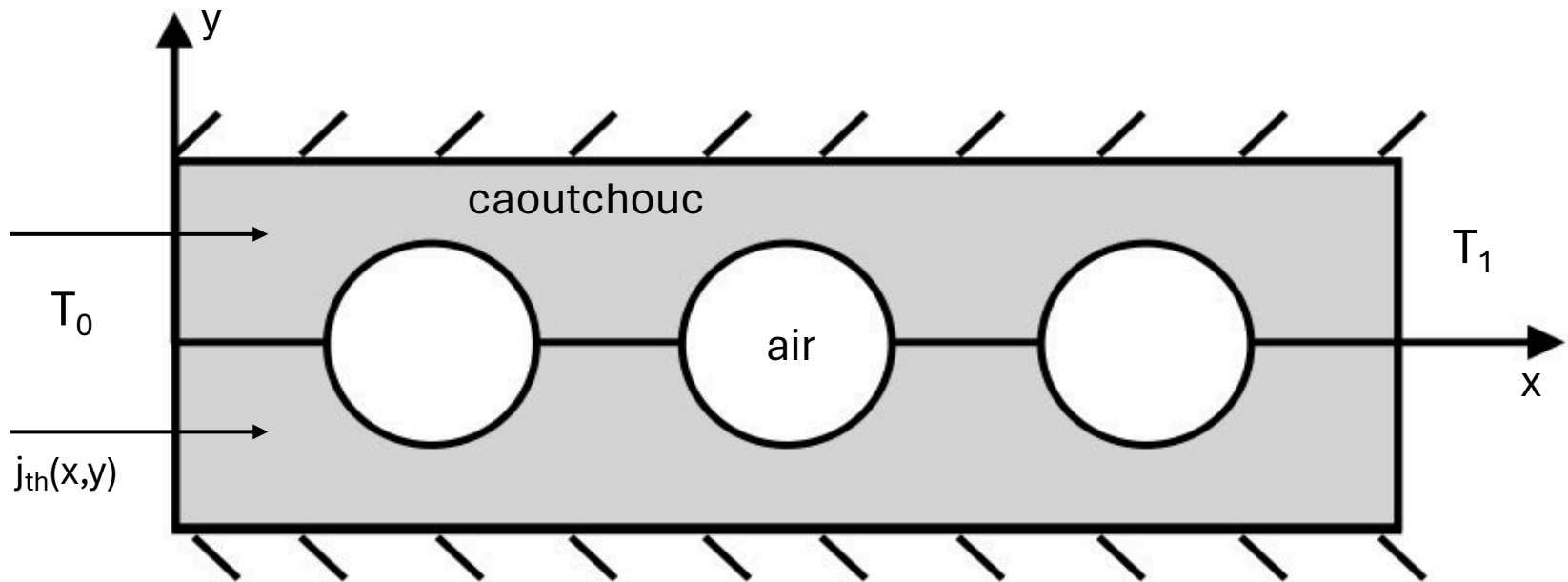
Solution technique :

Néoprène à cellule fermé = néoprène moussé



2^{ème} approche en 2D

Inclusions d'air dans le matériau néoprène :



Idee : étudier le flux et la résistance thermique selon la disposition et les caractéristiques des bulles

Mise en équation

Loi locale de la conservation d'énergie :

$$\frac{\partial u_v}{\partial t} + \operatorname{div}(\overrightarrow{j_{th}}) = 0$$

Equation de la chaleur adimensionnée 2D :

$$\tilde{\mu}\tilde{c} \frac{\partial \theta}{\partial \tau} = \frac{\partial}{\partial X} \left(\tilde{k} \frac{\partial \theta}{\partial X} \right) + \underbrace{\frac{\partial}{\partial Y} \left(\tilde{k} \frac{\partial \theta}{\partial Y} \right)}$$

Terme en plus
du au 2D

Disposition de bulles sur une grille

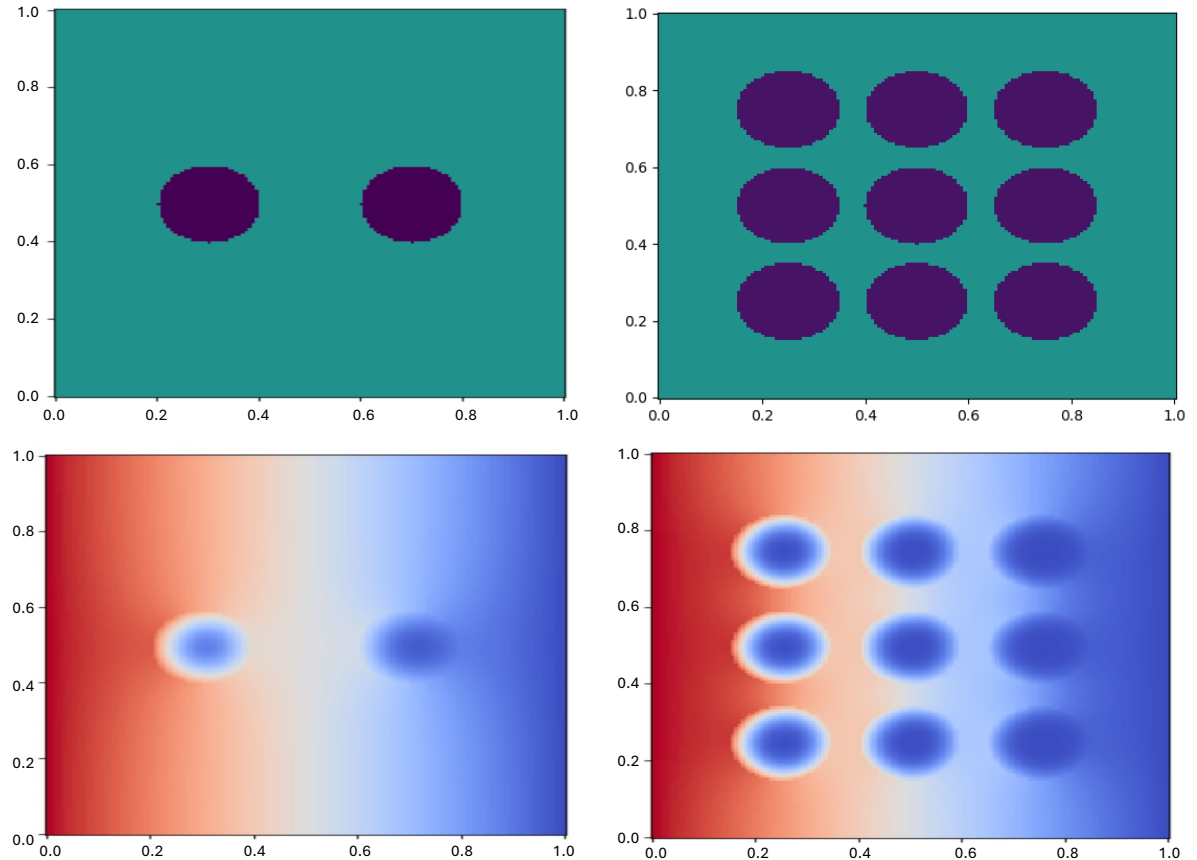
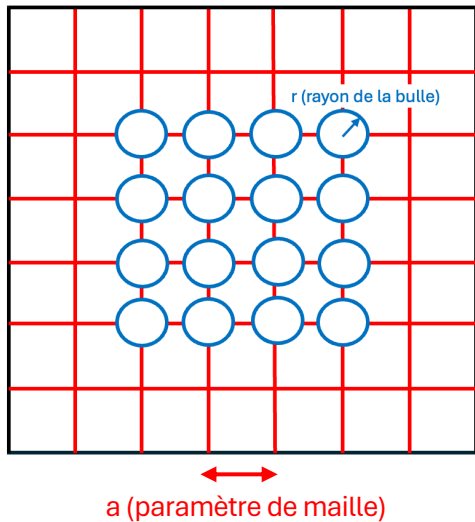


Figure : implémentation python des inclusions des bulles d'air

Régime Stationnaire en 2D

Régime stationnaire :

$\text{div}(\vec{j}_{th}) = 0 \Rightarrow \vec{j}_{th}$ à *flux conservatif* (flux indépendant de la surface)

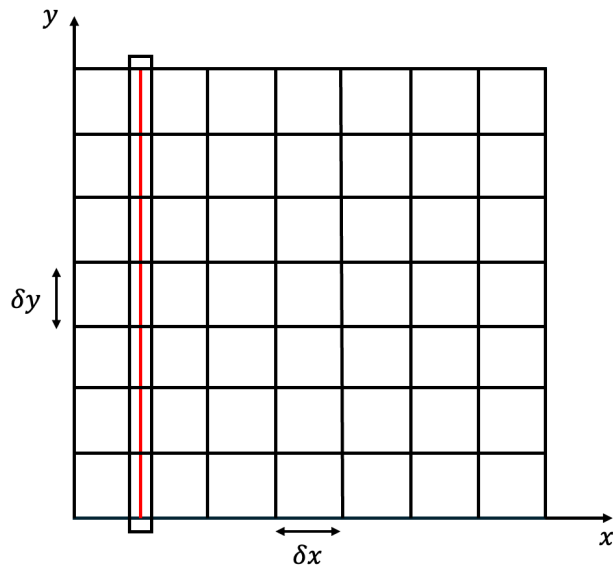
$$\phi = \int_0^L \vec{j}_{th} \cdot \vec{u}_x dy$$

Résolution numérique



$$\phi = \sum_{j=0}^{N_y} \left(\frac{\theta_{N_x-1,j} - \theta_{N_x,j}}{\delta x} \right) k \delta y$$

```
flux=-np.sum(kappa[:,1:]* (theta_e[:,1:,3999]-theta_e[:, :-1,3999]),axis=0)
```



Calcul du flux en intégrant selon différentes colonnes :

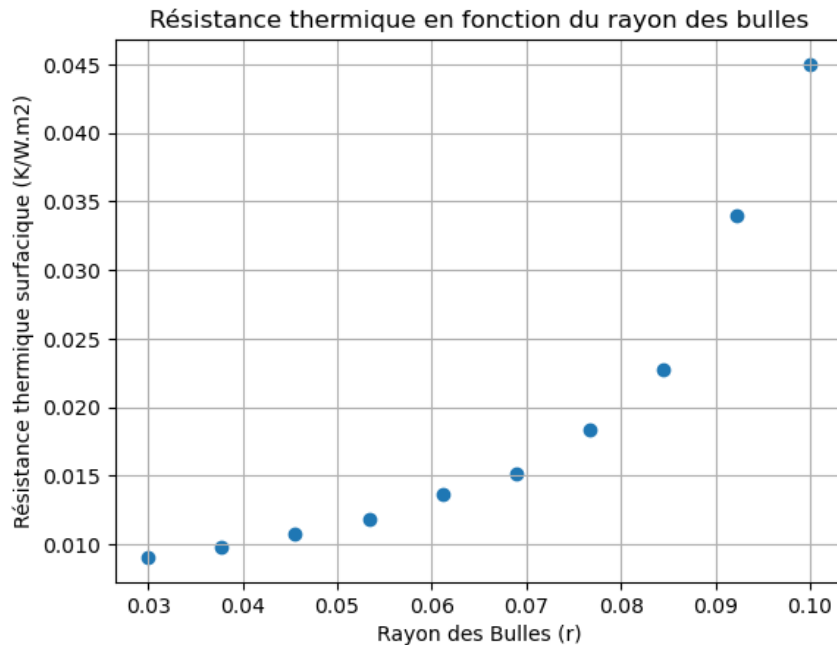
$$\text{Variance} = 3,7 \times 10^{-6}$$

$$\text{Ecart - Type} = 1,9 \times 10^{-3}$$

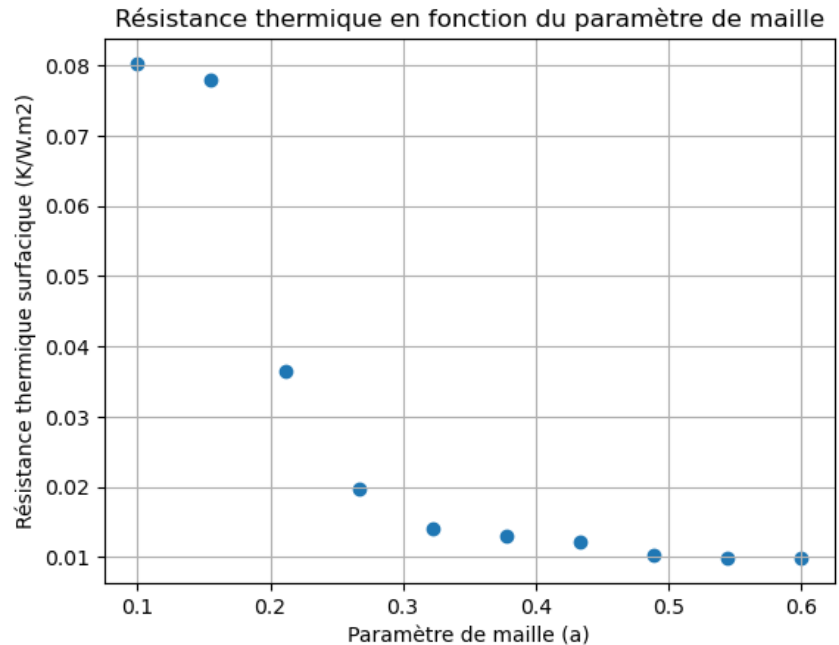
Validité du calcul : hypothèse = flux identique peu importe la colonne

Analyse de la résistance thermique

Effet des variations des paramètres géométriques sur la résistance thermique surfacique



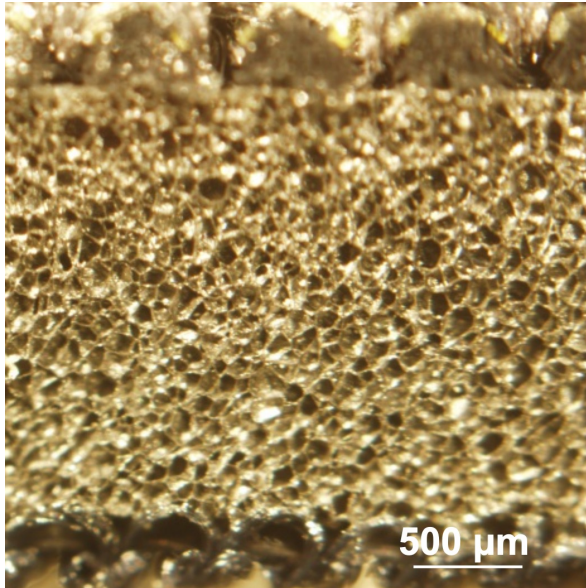
Variation du rayon des bulles



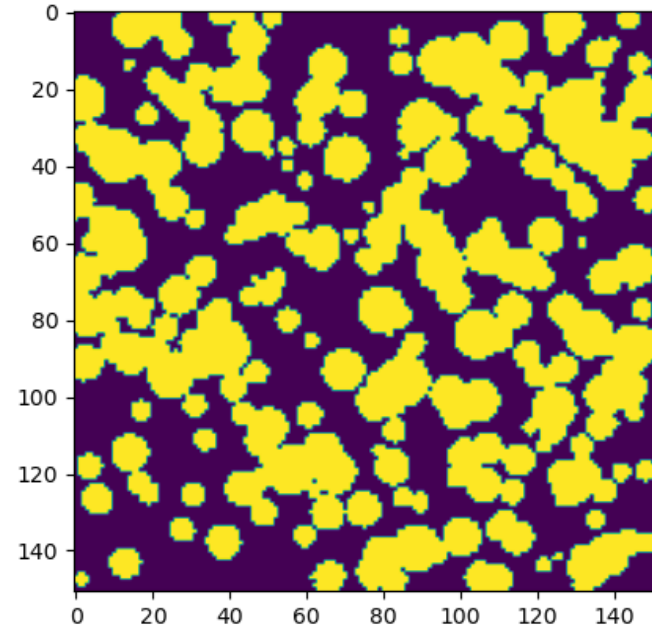
Variation du paramètre de maille

Approche statistique

Contraintes mécaniques et physiques :



Taux d'occupation proche de 0,8
Rayon d'une bulle proche de 0,15 mm



Position des bulles : loi uniforme
Rayon des bulles : loi gaussienne

Calcul du taux d'occupation :

```
fraction=x*0  
fraction[cond]=1  
np.sum(np.sum(fraction,axis=0),axis=0)/(Nx*Ny)
```

Annexe 6

Analyse du modèle

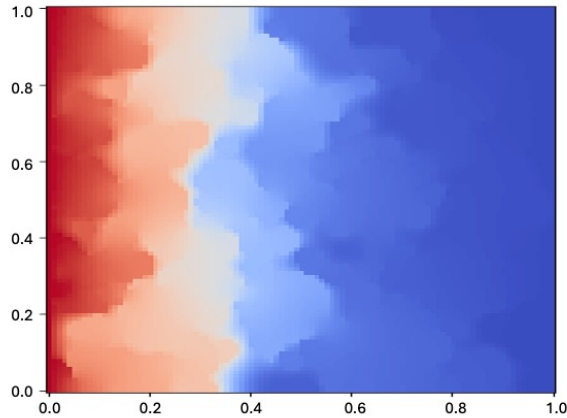
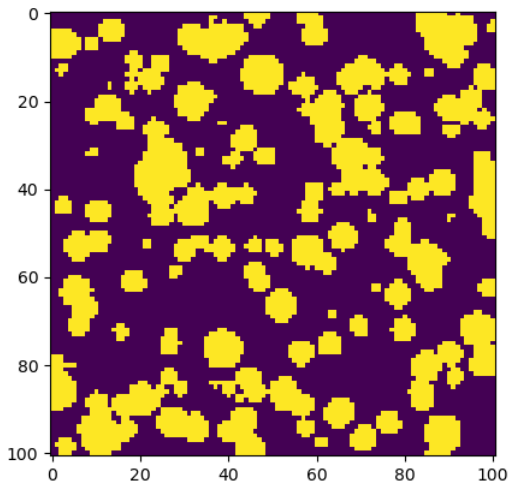


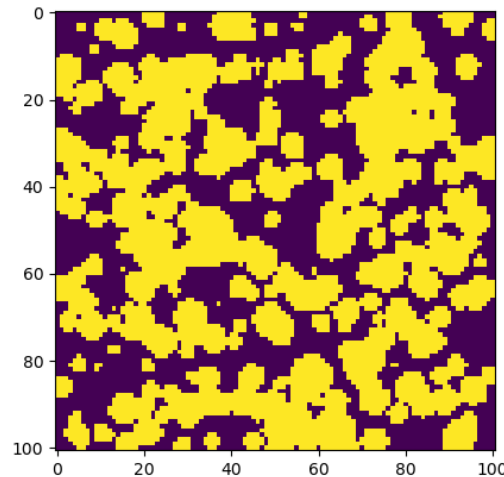
Figure : Visualisation de la diffusion dans le matériau non homogène

Effet de la variation du taux d'occupation de l'air sur la résistance thermique surfacique :



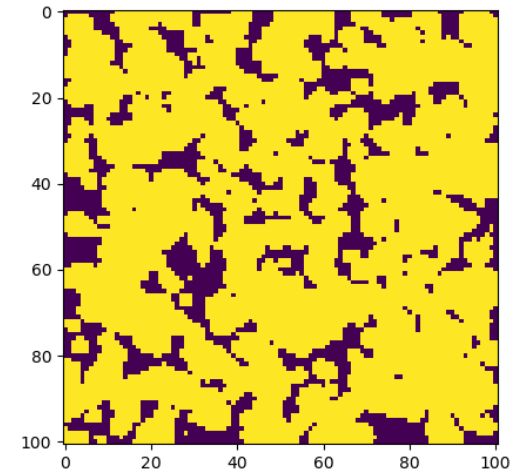
Taux d'occupation: 0,35

$$R_{th,surfacique} = 0,016 \text{ m}^2.K.W^{-1}$$



Taux d'occupation: 0,55

$$R_{th,surfacique} = 0,023 \text{ m}^2.K.W^{-1}$$

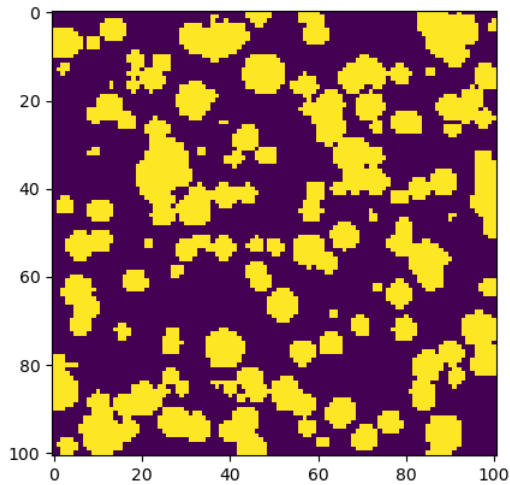


Taux d'occupation: 0,80

$$R_{th,surfacique} = 0,051 \text{ m}^2.K.W^{-1}$$

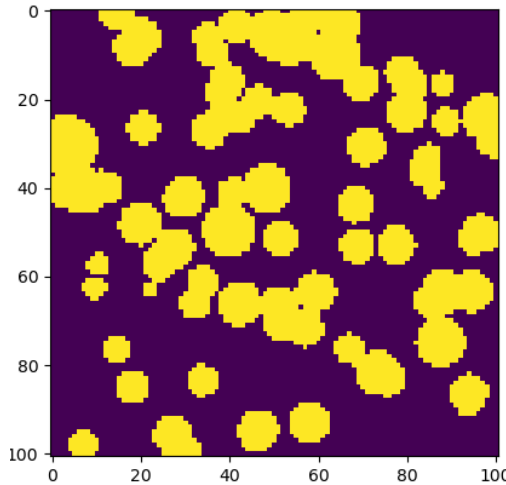
Analyse du modèle

Evolution de la résistance thermique surfacique en fonction de la taille des bulles (avec un taux d'occupation fixé à 0,35)



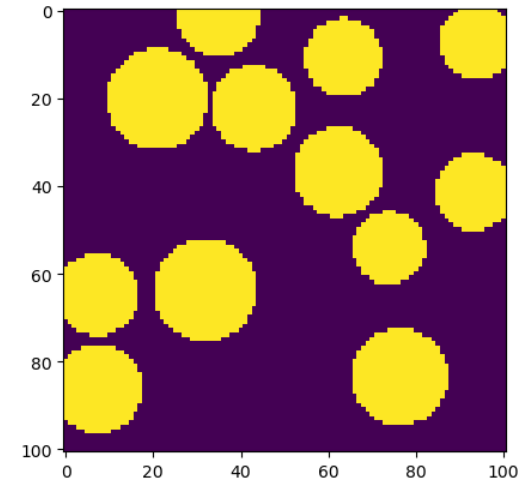
Taille moyenne des bulles: 0,075 mm

$$R_{th,surfacique} = 0,016 \text{ m}^2.K.W^{-1}$$



Taille moyenne des bulles: 0,15mm

$$R_{th,surfacique} = 0,016 \text{ m}^2.K.W^{-1}$$



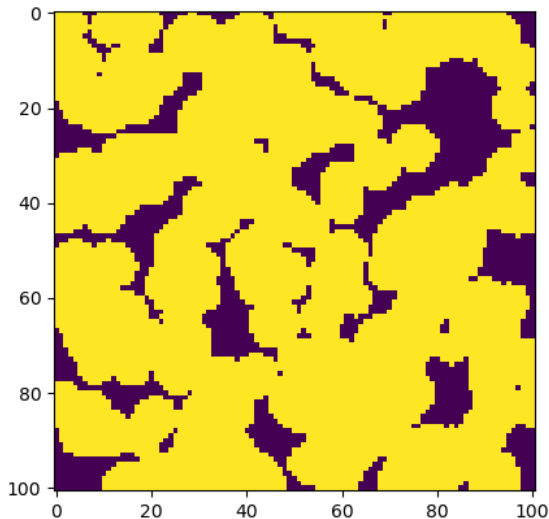
Taille moyenne des bulles: 0,3mm

$$R_{th,surfacique} = 0,017 \text{ m}^2.K.W^{-1}$$

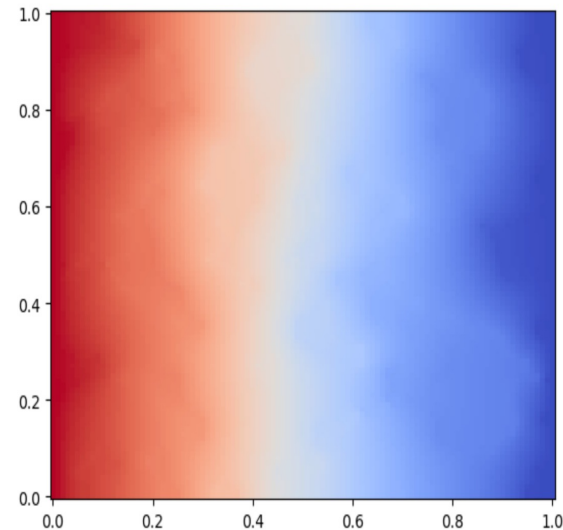
La résistance thermique est indépendante du rayon des bulles tant que le taux d'occupation est constant

Simuler la situation réelle

Caractéristiques techniques du néoprène (épaisseur 3mm)	Valeur numérique
Taux d'occupation de l'air	$\approx 0,8$
Rayon moyen d'une bulle	0,15 mm
Résistance thermique surfacique	$0,0526 \text{ m}^2.\text{K.W}^{-1}$



Taux d'occupation: 0,82

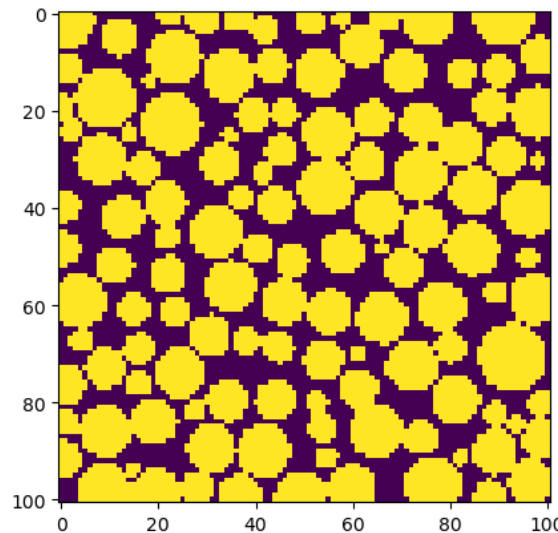


$$R_{\text{th,surfacique}} = 0,0509 \text{ m}^2.\text{K.W}^{-1}$$
$$\approx 0,0526 \text{ m}^2.\text{K.W}^{-1}$$

Conclusion

Nous sommes parvenus à créer un modèle assez complexe pour retrouver numériquement une valeur approchée de la résistance thermique du néoprène.

On pourrait tout de même envisager d'améliorer encore le modèle en s'assurant que les bulles ne se superposent plus.



ATTENTION à la complexité de l'algorithme !

Annexe 1

```
1  deltax=1e-2
2  deltat=.5*deltax**2
3
4  d=1. # durée de l'évolution
5
6  Nx=int(1/deltax) # nombre d'intervalles sur l'axe spatial
7  Nt=int(d/deltat) # nombre d'intervalles de temps
8  x=np.linspace(0,1,Nx+1)
9  t=np.linspace(0,d,Nt+1)
10
11 theta=np.zeros((Nx+1,Nt+1)) #initialisation du champ de temperature : premier indice (espace) et second (temps)
12
13 # valeur de mu,c,kapa tilde
14 L1=[1 for i in range(Nx//2)]
15 L2=[0.5 for i in range(Nx//2+1)]
16 kapa=L1+L2
17 mu=[1 for i in range(Nx+1)]
18 c=[1 for i in range(Nx+1)]
19
20
21 #condition initiale :
22 for i in range(Nx):
23     theta[i,0]=0
24
25 #conditions aux limites
26 for j in range(Nt):
27     theta[0,j]=0
28     theta[Nx,j]=1
29
30 # calcul du champ de température
31 for j in range(Nt):
32     for i in range(1,Nx):
33         theta[i,j+1]=(1/(c[i]*mu[i]))*(deltat/deltax)*(kapa[i+1]*((theta[i+1,j]-theta[i,j])/deltax)-kapa[i]
34                                     *(theta[i,j]-theta[i-1,j])/deltax)+theta[i,j]
35
36
37
```

```
1  N_visu=Nt//20#indice de l'instant auquel on veut visualiser le champ de température
2  plot(x,theta[:,N_visu], 'r-', linewidth=3, label='$\\theta$(Nt/20)') #visualisation du champ de température
3  plot(x,theta[:,Nt//10], 'b-', linewidth=3, label='$\\theta$(Nt/10)')
4  plot(x,theta[:,Nt//5], 'c-', linewidth=3, label='$\\theta$(Nt/5)')
5  plot(x,theta[:,Nt-1], 'g-', linewidth=3, label='$\\theta$(RS)')
6  legend(loc=2)
7  ylim([-0.1,1.1])
8  xlabel("$\\frac{x}{L}$", size=20); ylabel("$T-T_0/T_1-T_0$", size=20); title("thermalisation")
9  savefig("1D barre d'air")
```

Visualisation
des courbes

Annexe 2

Calcul de la résistance thermique surfacique équivalente :

```
1 kn=0.192
2 L=3e-3
3
4 # calcul du jth(adimensionné)en RS : (sous forme d'un tableau pour chaque valeur en fonction des x)
5 j=[0 for i in range (Nx)]
6 for i in range(Nx):
7     j[i] = -kapa[i]*(theta[i+1,Nt-1]-theta[i,Nt-1])/deltax
8 # en RS jth est constant (on prends quand même une valeur moyenne)
9 somme=0
10 for i in range(len(j)):
11     somme=somme+j[i]
12 jmoy=somme/len(j)
13
14 # calcul de la résistance thermique (non adimensionnée)
15 Rth=-L/(kn*jmoy)
16 print(Rth)
```

Annexe 3

Création de la maille en plaçant des bulles sur les nœuds :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Paramètres de la grille
5 Nx = 100
6 Ny = 100
7 deltax = 1. / Nx
8 deltax = 1. / Ny
9 deltat = .05 * (deltay ** 2)
10
11 d = 2 # Durée de l'évolution
12 Nt = int(d / deltat) # Nombre de pas de temps
13 Ne = Nt // 100
14
15 xx = np.linspace(0, 1, Nx + 1)
16 yy = np.linspace(0, 1, Ny + 1)
17 t = np.linspace(0, 1, Ne + 1)
18 y, x = np.meshgrid(yy, xx)
19
20 theta=np.zeros((Nx+1,Ny+1))
21 theta_e=np.zeros((Nx+1,Ny+1,Ne+1))
22
23 # Paramètres de maille et des bulles
24 a = 0.5 # Espacement entre les bulles
25 r = 0.1 # Rayon des bulles
26
27 # Calcul du nombre de bulles par ligne/colonne
28 num_bubbles_x = int(1 / a) + 1
29 num_bubbles_y = int(1 / a) + 1
30
31 # Calcul des décalages pour centrer la grille
32 offset_x = (1 - (num_bubbles_x - 1) * a) / 2
33 offset_y = (1 - (num_bubbles_y - 1) * a) / 2
34
35 # Initialisation des inclusions
36 inclusion = np.zeros((Nx + 1, Ny + 1), dtype=bool)
37
38 # Positionner les bulles sur une grille régulière centrée
39 for i in range(num_bubbles_x):
40     for j in range(num_bubbles_y):
41         x1 = offset_x + i * a
42         y1 = offset_y + j * a
43         inclusion |= (x - x1) ** 2 + (y - y1) ** 2 <= r ** 2
44
45 # Matrices de conductivité, masses volumiques et capacité thermiques massiques
46 kappa = np.ones((Nx + 1, Ny + 1))
47 mu = np.ones((Nx + 1, Ny + 1))
48 c = np.ones((Nx + 1, Ny + 1))
49
50 # Superposition des valeurs des inclusions
51 kappa[inclusion] = 0.15
52 mu[inclusion] = 1
53 c[inclusion] = 1
54
55 muc = mu * c
```

Annexe 4

Résolution de l'équation de la chaleur 2D et calcul de la résistance thermique surfacique équivalente

```
1 for j in range(Nt):
2     theta[1:-1,1:-1] += 1./muc[1:-1,1:-1]*(deltat/deltax*((theta[2:,1:-1]-theta[1:-1,1:-1])*kappa[2:,1:-1]/deltax
3                                     +(theta[: -2,1:-1]-theta[1:-1,1:-1])*kappa[1:-1,1:-1]/deltax)
4                                     +deltat/deltay*((theta[1:-1,2:]-theta[1:-1,1:-1])*kappa[1:-1,2:]/deltay
5                                     +(theta[1:-1,: -2]-theta[1:-1,1:-1])*kappa[1:-1,1:-1]/deltay)
6     #C.L.
7     theta[0,:]=theta[1,:]
8     theta[Nx,:]=theta[Nx-1,:]
9     theta[:,0]=1
10    theta[:,Ny]=0
11    if j%100==0:
12        theta_e[:,j//100]=theta[:,j//100]
13    flux=-np.sum(kappa[:,1:]*(theta_e[:,1:,3999]-theta_e[:,1:-1,3999]),axis=0)
```

Annexe 5

Tracé de la résistance thermique en fonction du rayon des bulles / même code adapté pour faire varier le paramètre de maille :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Paramètres de la grille
5 Nx = 100
6 Ny = 100
7 deltax = 1. / Nx
8 deltay = 1. / Ny
9 deltat = .05 * (deltay ** 2)
10
11 d = 2 # Durée de l'évolution
12 Nt = int(d / deltat) # Nombre de pas de temps
13 Ne = Nt // 100
14
15 # Définir une plage de valeurs pour le paramètre de maille a
16 r_values = np.linspace(0.03, 0.1, 10) # Par exemple, de 0.1 à 0.9 avec 9 valeurs
17 a=0.2
18 # Liste pour stocker les résistances thermiques calculées
19 rth_values = []
20
21 # Pour chaque valeur de paramètre de maille
22 for r in r_values:
23     # Grille de points
24     xx = np.linspace(0, 1, Nx + 1)
25     yy = np.linspace(0, 1, Ny + 1)
26     y, x = np.meshgrid(yy, xx)
27
28     # Calcul des décalages pour centrer la grille en fonction de a
29     num_bubbles_x = int(1 / a) + 1
30     num_bubbles_y = int(1 / a) + 1
31     offset_x = (1 - (num_bubbles_x - 1) * a) / 2
32     offset_y = (1 - (num_bubbles_y - 1) * a) / 2
33
34     # Initialisation des inclusions
35     inclusion = np.zeros((Nx + 1, Ny + 1), dtype=bool)
36
37     # Positionner les bulles sur une grille régulière centrée
38     for i in range(num_bubbles_x):
39         for j in range(num_bubbles_y):
40             x1 = offset_x + i * a
41             y1 = offset_y + j * a
42             inclusion |= (x - x1) ** 2 + (y - y1) ** 2 <= r ** 2
43
44     # Matrices de conductivité, masses volumiques et capacité thermiques massiques
45     kappa = np.ones((Nx + 1, Ny + 1))
46     mu = np.ones((Nx + 1, Ny + 1))
47     c = np.ones((Nx + 1, Ny + 1))
48
49     # Superposition des valeurs des inclusions
50     kappa[inclusion] = 1.e-1
51     mu[inclusion] = 1.
52     c[inclusion] = 1.
53
54     muc = mu * c
55
56     # Initialisation du champ de température
57     theta = np.zeros((Nx + 1, Ny + 1))
58
59     # Simulation de la diffusion de chaleur
60     for j in range(Nt):
61         theta[1:-1,1:-1] += (1. / muc[1:-1,1:-1] * (deltat / deltax *
62             ((theta[2:,1:-1] - theta[1:-1,1:-1]) * kappa[2:,1:-1] / deltax +
63             (theta[:-2,1:-1] - theta[1:-1,1:-1]) * kappa[1:-1,1:-1] / deltax) +
64             deltat / deltax *
65             ((theta[1:-1,2:] - theta[1:-1,1:-1]) * kappa[1:-1,2:] / deltax +
66             (theta[1:-1,-2] - theta[1:-1,1:-1]) * kappa[1:-1,-1] / deltax)))
67
68     # Boundary conditions
69     theta[0, :] = theta[1, :]
70     theta[Nx, :] = theta[Nx-1, :]
71     theta[:, 0] = 1 # Adiabatic wall on the left
72     theta[:, Ny] = 0 # Adiabatic wall on the right
73
74     if j % 100 == 0:
75         theta_e[:, :, j//100] = theta[:, :]
76
77         j_value = -np.sum(kappa[:, 1:] * (theta_e[:, 1:, Ne] - theta_e[:, :-1, Ne]), axis=0)
78         j_mean = np.mean(j_value)
79
80     # Calcul de la résistance thermique pour cette valeur de a
81     rth = L / (kc * j_mean)
82
83     # Ajouter la résistance thermique à la liste
84     rth_values.append(rth)
85
86 plt.scatter(r_values, rth_values, marker='o')
87 plt.xlabel('Rayon des Bulles (r)')
88 plt.ylabel('Résistance thermique surfacique (K/W.m2)')
89 plt.title('Résistance thermique en fonction du rayon des bulles')
90 plt.grid(True)
91 plt.show()
```

Annexe 6

Inclusion aléatoire des bulles

```
1 rv1=st.norm(loc=.025,scale=0.01)
2 rv2=st.uniform(0,1)
3
4 Nx=100;Ny=100
5 deltax=1./Nx;deltay=1./Ny
6 deltat=.05*(deltay**2)
7
8 d=.2 # durée de l'évolution
9 Nt=int(d/deltat) # nombre de pas de temps
10
11
12 xx=np.linspace(0,1,Nx+1)
13 yy=np.linspace(0,1,Ny+1)
14 y,x=np.meshgrid(yy,xx)
15
16
17 cond=x<-1
18 Nb=200
19 yc=rv2.rvs(Nb)
20 xc=rv2.rvs(Nb)
21 rc=rv1.rvs(Nb)
22
23 #plot(xc,yc,'ro')
24 savefig("placement des points")
25
26 for i in range(Nb):
27     bulle=(x-xc[i])**2+(y-yc[i])**2<=rc[i]**2
28     cond=np.bitwise_or(cond,bulle)
29 imshow(cond)
30 savefig("approche statistique")
31 fraction=x*0
32 fraction[cond]=1
33 np.sum(np.sum(fraction,axis=0),axis=0)/(Nx*Ny)
34
```

```
1 deltax=1./Nx;deltay=1./Ny
2 deltat=.05*(deltay**2)
3
4 d=2# durée de l'évolution
5 Nt=int(d/deltat) # nombre de pas de temps
6 Ne=Nt//100
7
8 xx=np.linspace(0,1,Nx+1)
9 yy=np.linspace(0,1,Ny+1)
10 t=np.linspace(0,1,Ne+1)
11 y,x=np.meshgrid(yy,xx)
12 #x,y=np.meshgrid(xx,yy)
13
14 theta=np.zeros((Nx+1,Ny+1))
15 theta_e=np.zeros((Nx+1,Ny+1,Ne+1))
16
17 mu=x*0+1
18 c=x*0+1
19 kappa=x*0+1
20 #on superpose au tableau
21 #des valeurs le tableau inclusion
22 kappa[cond]=0.13
23 mu[cond]=1.
24 c[cond]=1
25
26
27 muc=mu*c
```